GA4 Attribution Models — Usage Guide (v2.0)

Repository: github.com/GlitchG/ga4-attribution-models
License: MIT
Dataform version: 3.x
Last updated: 2026-05-05

Table of Contents

---

1. Quick Start

Prerequisites

- A Google Cloud project with BigQuery enabled
- A GA4 BigQuery export (public or private)
- Dataform CLI installed (`npm install -g @dataform/cli`)
- BigQuery permissions: `bigquery.dataEditor` , `bigquery.jobUser`

One-minute setup

```
 # Clone the repository
git clone https://github.com/GlitchG/ga4-attribution-models.git
cd ga4-attribution-models

# Install dependencies
npm install
```

```
# Create credentials (service account with BigQuery access)
gcloud auth application-default login   # or use a service account JSON
echo '{"projectId":"YOUR_PROJECT","location":"US","credentials":"BASE64_ENCODED_JSON"}' > .df-cre

# Configure your GA4 dataset
# Edit workflow_settings.yaml:
#   vars:
#     ga4_project: "your-project"
#     ga4_dataset: "analytics_xxx"
#     start_date: "20240101"
#     end_date: "20240131"

# Compile and run
dataform compile
dataform run
```

Public sample (free, no setup)

The repo defaults to `bigquery-public-data.ga4_obfuscated_sample_ecommerce`. Just run:

```
dataform compile
dataform run --default-database=YOUR_PROJECT
```

---

2. Architecture Overview

Session-based, not event-based. The pipeline:

1. **Extracts sessions** using configurable source extraction mode (`event_params` , `session_stslc` , `collected` , or `auto`)
2. **Extracts all configured conversion events** with full ecommerce payload
3. **Builds journeys** — for each conversion, finds all sessions within the lookback window
4. **Unnests paths** into row-level format for attribution calculation
5. **Runs 8 attribution models** in parallel
6. **Unifies results** into a single mart table with cross-model comparison

```
GA4 events_* → stg_ga4_sessions + stg_ga4_conversions
                    ↓
       int_attribution_journeys (path building)
                    ↓
       int_attribution_path_rows (row-level unnest)
                    ↓
   ┌──────────────────────────────────────────────────┐
   │  first_click / last_click / last_non_direct_click │
   │  linear / time_decay / u_shape / position_weighted │
   │  data_driven_bqml (ML model)                      │
```

```
    └──────────────────────────────────────────────────┘
                          ↓
        attribution_mart + cross_channel_comparison
```

Key design decisions

Decision	Rationale
Session-based	GA4's session concept aligns with marketing touchpoints; event-level is too granular
Multi-conversion	A single user can convert multiple times; each conversion gets its own journey
30-day lookback	Configurable; default matches GA4's default attribution window
Auto source resolution	Works across GA4 export versions without schema changes

---

3. Configuration

3.1 Conversion events (`includes/conversion_config.js`)

```
const CONVERSION_EVENTS = [
  {
    event: 'purchase',
    value_mode: 'revenue',   // Uses purchase_revenue_in_usd
    description: 'Completed purchase'
  },
  {
    event: 'begin_checkout',
    value_mode: 'count',     // No revenue; counts conversions only
    description: 'Checkout started'
  },
  {
    event: 'add_to_cart',
    value_mode: 'count',
    description: 'Item added to cart'
  }
];
```

Value modes: - `revenue` — Uses `purchase_revenue_in_usd` (or event-specific revenue field) - `fixed` — Assigns a fixed value per conversion (configure in `getValueExpr()`) - `count` — No monetary value; `attributed_value_usd` will be NULL

To add a new conversion: 1. Append to `CONVERSION_EVENTS` array 2. Choose `value_mode` 3. Recompile — no SQL changes needed

3.2 Channel grouping (`includes/channel_grouping.js`)

17-channel taxonomy:

1. Cross-network
2. Paid Search Brand
3. Paid Search Non-Brand
4. Paid Shopping
5. Paid Social
6. Paid Video
7. Display
8. Organic Search
9. Organic Shopping
10. Organic Social
11. Organic Video
12. Email
13. SMS
14. Affiliate
15. Audio
16. Mobile Push
17. Referral
18. Direct

Brand vs Non-Brand split: Edit `BRAND_TERMS_REGEX` in `includes/constants.js` :

```
const BRAND_TERMS_REGEX = "(yourbrand|yourcompany|yourproduct)";
```

This regex is used in the `channelGrouping()` function. If `source` matches (case-insensitive), the channel becomes "Paid Search Brand"; otherwise "Paid Search Non-Brand".

3.3 Source extraction mode (`workflow_settings.yaml`)

```
vars:
  source_extraction_mode: "event_params"   # Options: auto, event_params, session_stslc, collecte
```

Mode	Use when	Description
`event_params` (default)	Any export	Extracts source/medium from event-level `event_params` . Works on all GA4 exports.
`session_stslc`	Post-2024-07 exports	Uses `session_traffic_source_last_click` (GA4 UI-native logic).
`collected`		

Mode	Use when	Description
	Post-2023-06 exports	Uses `collected_traffic_source` (manual override values).
`auto`	Post-2024-07 exports	`COALESCE(session_stslc, collected, event_params)` — falls through gracefully.

Important: The public sample dataset (`bigquery-public-data.ga4_obfuscated_sample_ecommerce`) is from 2020 and does NOT have `session_traffic_source_last_click` or `collected_traffic_source` . Use `event_params` (default) for this dataset.

3.4 Lookback window (`workflow_settings.yaml`)

```
vars:
  lookback_days: "30"
```

The lookback is **exact**: the pipeline filters sessions where `session_start >= conversion_ts - lookback_days` . Additionally, `_TABLE_SUFFIX` is extended by `lookback_days` to ensure all relevant partitions are scanned.

3.5 Privacy / consent mode v2 (`workflow_settings.yaml`)

```
vars:
  exclude_modeled_events: "true"   # Set to "true" to exclude modeled events
```

When enabled, sessions with `privacy_info.uses_transient_token = 'Yes'` are excluded. All privacy fields are passed through to staging and attribution tables for auditability.

Note: The public sample dataset (2020) does not contain consent mode v2 fields. These will be NULL.

---

4. Running the Pipeline

Full run (all models)

```
dataform run --default-database=your-project
```

Run specific tags

```
 # Staging only
dataform run --tags=staging

# Attribution models only (skip ML)
dataform run --tags=attribution,model
```

```
# BQML training only
dataform run --tags=ml

# Dashboard views only
dataform run --tags=dashboard
```

Full refresh (rebuild everything)

```
dataform run --full-refresh
```

Schedule in production

Use Dataform's built-in scheduling (in the Dataform UI) or Cloud Scheduler:

```
# Example: daily at 06:00 UTC
gcloud scheduler jobs create http daily-attribution \
  --schedule="0 6 * * *" \
  --uri="https://dataform.googleapis.com/v1beta1/projects/YOUR_PROJECT/locations/us/repositories/
  --http-method=POST \
  --message-body='{"compilationResult":"projects/YOUR_PROJECT/locations/us/repositories/ga4-attri
```

---

5. Understanding the Output

5.1 `attribution_mart`

The unified output table. One row per touchpoint per conversion per model.

Column	Description
`user_pseudo_id`	GA4 user identifier
`conversion_id`	Unique conversion identifier
`conversion_ts`	Timestamp of conversion event
`conversion_event`	Event name (e.g. `purchase`, `begin_checkout`)
`transaction_id`	Ecommerce transaction ID (if available)
`conversion_value_local`	Revenue in local currency
`conversion_value_usd`	Revenue in USD
`path_length`	Number of sessions in the journey

Column	Description
`source` / `medium` / `campaign` / `channel`	Touchpoint dimensions
`model`	Attribution model name
`attributed_credit`	Fraction of credit (sums to 1.0 per conversion)
`attributed_value_usd`	Credit × conversion_value_usd
`attributed_value_local`	Credit × conversion_value_local

Example query — top channels by revenue (last-click):

```
SELECT
  channel,
  SUM(attributed_value_usd) AS revenue,
  COUNT(DISTINCT conversion_id) AS conversions
FROM `your-project.attribution_models.attribution_mart`
WHERE model = 'last_click'
  AND conversion_event = 'purchase'
GROUP BY channel
ORDER BY revenue DESC;
```

5.2 `cross_channel_comparison`

Pre-aggregated for dashboards. One row per model × conversion_event × channel.

Column	Description
`model`	Attribution model
`conversion_event`	Conversion event name
`channel`	Channel
`attributed_conversions`	Number of conversions
`total_value_local`	Sum of attributed local currency
`total_value_usd`	Sum of attributed USD
`avg_path_length`	Average path length for this segment

5.3 `int_attribution_journeys`

Raw journey data for custom analysis.

```
 -- Find the longest customer journeys
SELECT *
FROM `your-project.intermediate.int_attribution_journeys`
ORDER BY path_length DESC
LIMIT 10;
```

---

6. Attribution Models Explained

6.1 Rule-based models

Model	Logic	Best for
First Click	100% to first session	Brand awareness campaigns
Last Click	100% to last session	Direct response, simple funnels
Last Non-Direct Click	100% to last non-Direct session; falls back to Direct if none	Default GA4 logic
Linear	Equal credit to all sessions	Long consideration cycles
Time Decay	Credit decays with 7-day half-life	Recent-touch bias
U-Shape	40% first, 40% last, 20% middle	Balanced first/last emphasis
Position Weighted	50% first, 30% last, 20% middle	Data-driven heuristic proxy

6.2 Data-driven (BQML)

Trains a logistic regression model on: - **Positive class:** Users who converted (binary channel flags from their journey) - **Negative class:** Users who had sessions but did NOT convert

Features: `conversion_event` (categorical) + 17 binary channel flags.

Credit assignment: For each touchpoint in a converted journey, the model's predicted conversion probability is compared with and without that channel. The difference is the marginal contribution.

Known limitations: - Single model trained on all conversion events. If funnel stages have very different channel effects, consider per-conversion-event models (v2.1 roadmap). - Requires sufficient training data (recommended: >1,000 conversions, >10,000 non-converters). - Model training can take 2–5 minutes.

---

7. Troubleshooting

7.1 Compilation errors

Error	Cause	Fix
`Unrecognized name: session_traffic_source_last_click`	Using `session_stslc` mode on an old GA4 export	Switch to `event_params` mode
`No actions to run`	All tables already exist and are up-to-date	Use `--full-refresh` or delete tables
`Concurrent model update`	BQML model is being trained by another job	Wait or `DROP MODEL IF EXISTS`
`Permission denied`	Service account lacks BigQuery roles	Grant `bigquery.dataEditor` + `bigquery.jobUser`

7.2 Runtime errors

Error	Cause	Fix
`attr_data_driven_train` fails with "empty training data"	`int_attribution_path_rows` has no rows	Check `stg_ga4_conversions` has data; verify `_TABLE_SUFFIX` range
`int_attribution_journeys` is empty	No conversions in the date range	Extend `start_date` / `end_date` ; check conversion events are configured
Attribution credit sums to <1.0	Path-length edge case in U-shape/position_weighted	Already fixed in v2.0; update if on older version
`last_non_direct_click` drops conversions	Logic bug with `session_position_desc`	Already fixed in v2.0; update if on older version
Channel shows as "source / medium"	Catch-all triggered — unknown source/medium combination	Add channel mapping in `channel_grouping.js`

7.3 Data quality issues

Symptom	Diagnosis	Fix
All sessions are "Direct"	Source extraction mode doesn't match export schema	Check GA4 export version; switch mode
0 conversions	Wrong `event_name` in config	Verify event names in `conversion_config.js` match GA4
Revenue is NULL	`value_mode` set to `count`	Change to `revenue` in `conversion_config.js`
Duplicate sessions	Missing `ROW_NUMBER()` dedup	Already handled in `stg_ga4_sessions`
Very long paths (>20 sessions)	Bot traffic or returning users	Add bot filtering in `stg_ga4_sessions` WHERE clause

7.4 Performance issues

Symptom	Fix
Pipeline runs >30 minutes	Reduce `lookback_days`; narrow date range; use incremental models
High BigQuery costs	Use `_TABLE_SUFFIX` partitioning (already implemented); avoid full refreshes
BQML training fails	Ensure training table has >100 rows; check for NULL features

---

8. Advanced Topics

8.1 Adding a custom channel

1. Edit `includes/channel_grouping.js`
2. Add a `WHEN` clause before the `ELSE`:

```
WHEN COALESCE(${sourceExpr}, '') = 'my_custom_source' THEN 'Custom Channel'
```

1. Add to `getChannelList()`:

```
function getChannelList() {
 return [
   'Custom Channel',
   // ... existing channels
```

```
  ];
}
```

1. Recompile — all models update automatically.

8.2 Adding a custom attribution model

1. Create `definitions/attribution_models/attr_my_model.sqlx`
2. Copy structure from `attr_linear.sqlx`
3. Implement your credit-allocation logic
4. Add to `attribution_mart.sqlx` UNION ALL
5. Add to `cross_channel_comparison.sqlx`

8.3 Incremental models (production)

For daily runs with large datasets, convert staging tables to incremental:

```
config {
  type: "incremental",
  // ...
}

SELECT * FROM ...
${when(incremental(), `WHERE _TABLE_SUFFIX >= '${constants.START_DATE}'`)}
```

8.4 Custom conversion value

For `fixed` value mode, modify `includes/conversion_config.js`:

```
{
  event: 'sign_up',
  value_mode: 'fixed',
  fixed_value: 50.00,  // USD
  description: 'Newsletter signup'
}
```

Then update `getValueExpr()` to handle `fixed_value`.

8.5 Cross-project GA4 exports

If your GA4 data is in a different project:

```
vars:
  ga4_project: "client-project"
  ga4_dataset: "analytics_123456789"
```

The pipeline will read from `client-project` and write to your default database.

---

9. Validation & Testing

9.1 Built-in assertions

The pipeline includes 6 Dataform assertions:

1. `stg_ga4_conversions` unique key on `(user_pseudo_id, event_timestamp, event_name)`
2. `stg_ga4_conversions` row conditions: `conversion_value_usd >= 0`, `event_timestamp` valid
3. `int_attribution_journeys` unique key on `(user_pseudo_id, conversion_id)`
4. `int_attribution_journeys` row conditions: `conversion_value_usd >= 0`, `path_length >= 1`
5. `int_attribution_path_rows` unique key on `(user_pseudo_id, conversion_id, session_position_asc)`
6. `int_attribution_path_rows` row conditions: `ARRAY_LENGTH(path) = path_length`

9.2 Post-run validation queries

Run `validation/validation-queries.sql` after each execution:

```
bq query --use_legacy_sql=false < validation/validation-queries.sql
```

Key checks: - Credit sums to 1.0 per conversion - No unexpected conversion events - Revenue conservation (attributed total = raw total) - No duplicate sessions or journeys - Channel coverage matches taxonomy

9.3 Testing on the public sample

The public sample (`bigquery-public-data.ga4_obfuscated_sample_ecommerce`) is pre-configured as the default. Use it to verify your setup before connecting to private data.

Known public sample quirks: - Source/medium values are anonymised: `<Other>`, `(data deleted)` — mapped to `Direct` - No consent mode v2 fields (2020 dataset) - No `session_traffic_source_last_click` (use `event_params` mode)

---

10. Performance & Cost

Public sample (default config)

Stage	BigQuery Cost	Time
Staging (2 tables)	~2.9 GiB	~2 min

Stage	BigQuery Cost	Time
Intermediate (2 tables)	~0.15 GiB	~1 min
Attribution models (7 rule-based)	~0.08 GiB	~2 min
BQML training	~0.05 GiB	~3 min
Marts (2 tables)	~0.2 GiB	~1 min
Total	**~3.4 GiB**	**~9 min**

Cost estimate: ~$0.02 USD per run (on-demand pricing).

Production dataset (1B+ events)

Optimization	Impact
Incremental models	10–50x faster daily runs
Narrow date range	Linear cost reduction
Reduce `lookback_days`	Reduces partition scans
Materialised intermediate tables	Faster model re-runs

---

11. Migration from v1.x

Breaking changes

v1.x	v2.0	Action required
`purchase_revenue`	`conversion_value_local`	Update downstream queries
`purchase_revenue_in_usd`	`conversion_value_usd`	Update downstream queries
`attributed_revenue`	`attributed_value_usd`	Update downstream queries
`attributed_revenue_local`	`attributed_value_local`	Update downstream queries
Hardcoded `purchase` filter	Dynamic `conversion_config.js`	Edit `conversion_config.js`
8 channels	17 channels	Update cost module channel mapping

v1.x	v2.0	Action required
`source_resolution.js` inline	Mode-switchable via var	Set `source_extraction_mode`

Migration checklist

- [ ] Update downstream dashboards/queries with new column names
- [ ] Review `conversion_config.js` — ensure desired events are listed
- [ ] Set `source_extraction_mode` based on your GA4 export version
- [ ] Update cost module channel mappings (if using)
- [ ] Test on public sample before production
- [ ] Run validation queries

Appendix: File Reference

```
 includes/
  channel_grouping.js      -- 17-channel CASE logic
  conversion_config.js     -- Conversion events + value modes
  constants.js             -- Project vars + safe defaults
  source_resolution.js     -- Source extraction mode switch

definitions/
  staging/
    stg_ga4_sessions.sqlx      -- Session extraction
    stg_ga4_conversions.sqlx   -- Conversion extraction
  intermediate/
    int_attribution_journeys.sqlx     -- Path building
    int_attribution_path_rows.sqlx    -- Row-level unnest
  attribution_models/
    attr_*.sqlx                -- 8 attribution models
    attribution_mart.sqlx      -- Unified output
    cross_channel_comparison.sqlx  -- Aggregated comparison
  ml/
    attr_data_driven_train.sqlx   -- BQML training
  cost/
    attribution_with_roas.sqlx    -- Cost enrichment (disabled by default)
  dashboard/
    attribution_dashboard.sqlx    -- Dashboard views
  user_journey/
    path_analysis.sqlx         -- Top 100 paths

validation/
  validation-queries.sql     -- Post-run checks
```

For issues or feature requests, open a GitHub issue or submit a PR.